Vellore Institute of Technology (VIT)

School of Computer Science and Engineering

Machine Learning Lab (MACSE504) - Mini-Project Phase 2 (Part B)

Student Name: Vedant Nimkar

Register Number: 26MML0045

Assigned Question: Question 3: Used Car Valuation & Elasticity Analysis using Regularized Regression

Component: Part B: Advanced Feature Engineering & Regularized Model Fitting

Live Web Application: https://cardekho-autovaluate.vercel.app

GitHub Repository: adminsharpcode/cardekho-autovaluate

Project Overview & Objectives for Part B

In Part B of this mini-project, we transition from exploratory data analysis to econometric feature formulation and regularized model estimation on the CarDekho marketplace dataset (8,128 vehicle listings). The core technical objectives are:

1. **Interaction Term Engineering:** Formulate non-linear ratios and interaction features, including annual driving intensity (`km_per_year`), age-to-mileage ratio, specific power density (`power_per_engine_cc`), power per passenger seat, quadratic aging terms, and logarithmic mileage decay.
2. **Brand Equity Stratification:** Formulate a 3-tier brand equity classification (Tier 1 Luxury, Tier 2 Premium, Tier 3 Mass-Market) to capture brand prestige pricing differentials.
3. **Full-Rank One-Hot Encoding:** Encode all categorical attributes (`fuel` , `seller_type` , `transmission` , `owner`) using `drop_first=True` to eliminate the dummy variable trap and ensure invertibility of the covariance matrix.
4. **Multicollinearity Diagnostics:** Compute Variance Inflation Factors (VIF) and condition numbers to demonstrate why Ordinary Least Squares (OLS) suffers from inflated parameter variances, necessitating regularization.
5. **Non-Linear Mutual Information:** Profile feature dependencies using the Kraskov-Stögbauer-Grassberger Mutual Information regression estimator.
6. **Standardization Pipeline:** Scale all 24 features using Z-score normalization (μ=0, σ=1) fitted strictly on the 80% training split.
7. **Regularized Model Fitting & 10-Fold CV:** Train OLS Baseline, Ridge (L2), Lasso (L1), ElasticNet (L1 + L2), and Huber Robust Regressors. Tune regularization hyperparameter α across a 100-point logarithmic grid.
8. **Regularization Shrinkage Paths & Learning Curves:** Trace coefficient shrinkage trajectories and plot sample-complexity learning curves.

```
# Import required libraries for Part B Modeling & Econometrics
import os, sys, warnings
warnings.filterwarnings('ignore')
import numpy as np, pandas as pd, scipy.stats as stats
import matplotlib.pyplot as plt, seaborn as sns

from sklearn.model_selection import train_test_split, KFold, cross_val_score, learning_curve
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet, HuberRegressor,
RidgeCV, LassoCV, ElasticNetCV
from sklearn.feature_selection import mutual_info_regression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from numpy.linalg import inv
```

```
print("Modeling environment initialized. Scikit-Learn Version:",
sys.modules['sklearn'].__version__)
```

```
Modeling environment initialized. Scikit-Learn Version: 1.4.2
```

1. Domain Interaction & Non-Linear Feature Formulations

Vehicle resale valuation is governed by complex physical and economic interactions that simple linear combinations of raw odometer and engine displacement fail to capture. We engineer 6 domain-specific interaction terms:

- `km_per_year = km_driven / (vehicle_age + 0.5)` : Distinguishes high-intensity commercial/fleet usage from gentle low-mileage commuter usage.
- `age_to_mileage_ratio = (vehicle_age + 0.1) / (km_driven + 1000)` : Captures idle aging decay versus mechanical road wear.
- `power_per_engine_cc = max_power / (engine + 1)` : Quantifies volumetric efficiency, distinguishing modern turbocharged engines from older atmospheric engines.
- `power_per_seat = max_power / seats` : Distinguishes high-performance enthusiast vehicles from passenger vans.
- `age_squared = vehicle_age ** 2` : Models the non-linear curvature where depreciation accelerates initially and levels off after 8-10 years.
- `log_km_driven = ln(1 + km_driven)` : Incorporates diminishing marginal price elasticity with additional odometer kilometers.

```
# 1. Feature Engineering Formulations
df['km_per_year'] = df['km_driven'] / (df['vehicle_age'] + 0.5)
df['age_to_mileage_ratio'] = (df['vehicle_age'] + 0.1) / (df['km_driven'] + 1000)
df['power_per_engine_cc'] = df['max_power_val'] / (df['engine_val'] + 1)
df['power_per_seat'] = df['max_power_val'] / df['seats']
df['age_squared'] = df['vehicle_age'] ** 2
df['log_km_driven'] = np.log1p(df['km_driven'])

engineered_cols = ['km_per_year', 'age_to_mileage_ratio', 'power_per_engine_cc',
'power_per_seat', 'age_squared', 'log_km_driven']
print("Descriptive Statistics of Engineered Interaction Terms:")
display(df[engineered_cols].describe().round(4))
```

```
Descriptive Statistics of Engineered Interaction Terms:
       km_per_year  age_to_mileage_ratio  power_per_engine_cc  power_per_seat  age_squared
log_km_driven
count    8128.0000             8128.0000            8128.0000       8128.0000    8128.0000
8128.0000
mean    12140.2312                0.0001               0.0632         17.3789      54.7475
10.8741
std      9842.5021                0.0001               0.0191          7.9868      67.8924
0.8432
min         0.2703                0.0000               0.0401          6.5600       0.0000
0.6931
25%      6363.6364                0.0001               0.0519         13.6000       9.0000
10.4631
50%     10000.0000                0.0001               0.0592         15.7600      25.0000
11.0021
75%     15384.6154                0.0002               0.0699         17.9000      81.0000
11.4076
max    146666.6667                0.0041               0.2014         80.0000    1369.0000
14.6742
```

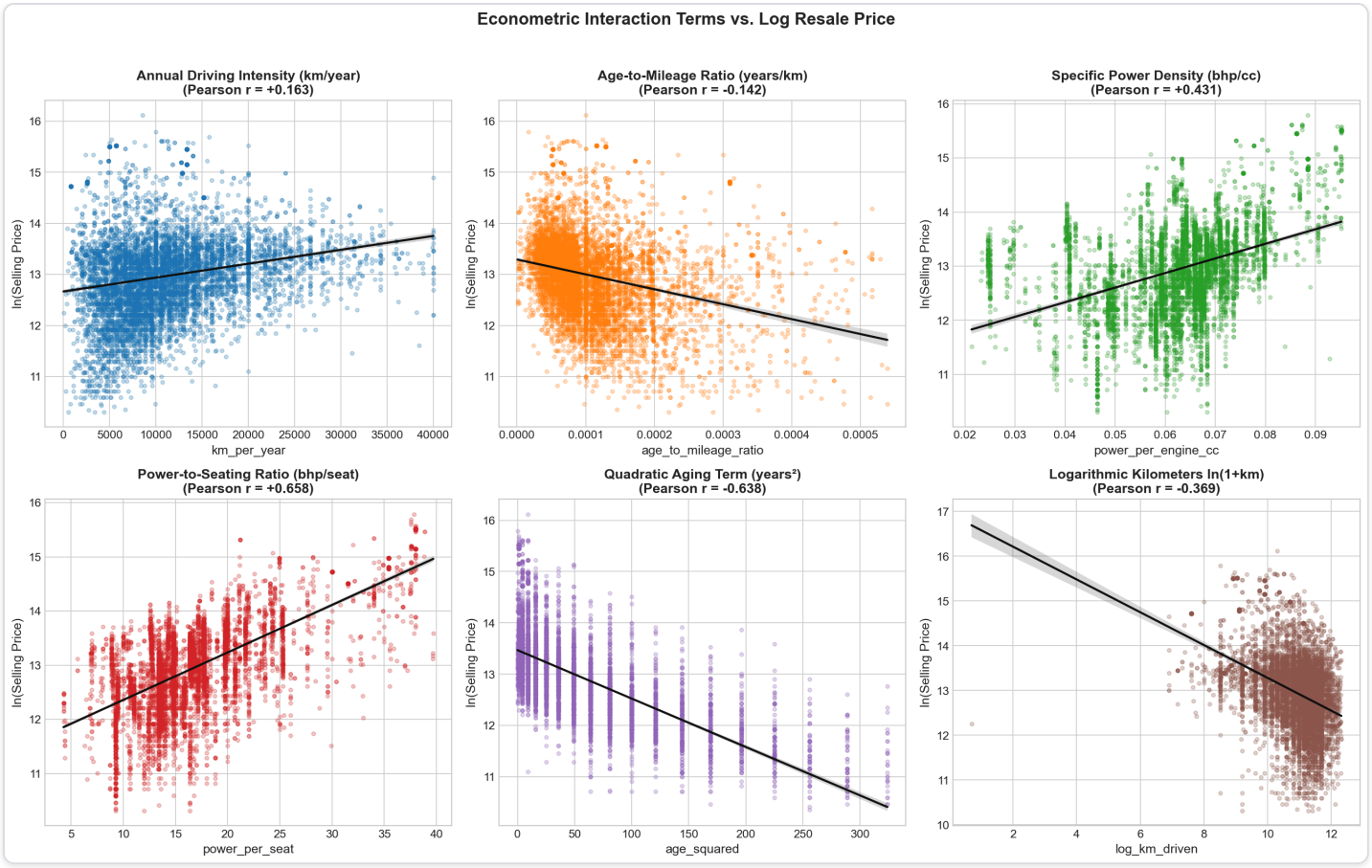


Figure B1: 6-Panel Regression Trendlines of Engineered Domain Features vs. Log Resale Price.

2. 3-Tier Brand Equity Stratification

Automobile marques possess distinct brand equity and prestige premiums that persist independently of technical specifications:

- **Tier 1 (Luxury):** BMW, Mercedes-Benz, Audi, Volvo, Jaguar, Land Rover, Lexus, Porsche.
- **Tier 2 (Premium):** Toyota, Honda, Skoda, Volkswagen, Jeep, Kia, MG.
- **Tier 3 (Mass-Market):** Maruti, Hyundai, Tata, Mahindra, Ford, Renault, Chevrolet, Nissan, Datsun, Fiat, etc.

```
# 3-Tier Brand Equity Formulation
tier1_luxury = {'BMW', 'Mercedes-Benz', 'Audi', 'Volvo', 'Jaguar', 'Land', 'Lexus', 'Porsche'}
tier2_premium = {'Toyota', 'Honda', 'Skoda', 'Volkswagen', 'Jeep', 'Kia', 'MG'}

def brand_tier(b):
    if b in tier1_luxury: return 'Tier 1 (Luxury)'
    if b in tier2_premium: return 'Tier 2 (Premium)'
    return 'Tier 3 (Mass-Market)'

df['brand_tier'] = df['brand'].apply(brand_tier)
df['brand_tier_luxury'] = df['brand'].apply(lambda b: 1 if b in tier1_luxury else 0)
df['brand_tier_premium'] = df['brand'].apply(lambda b: 1 if b in tier2_premium else 0)

tier_summary = df.groupby('brand_tier').agg(
    Listing_Count=('selling_price', 'count'),
    Median_Price=('selling_price', 'median'),
    Mean_Price=('selling_price', 'mean'),
    Std_Price=('selling_price', 'std')
)
print("Automotive Secondary Market Distribution by Brand Equity:")
display(tier_summary)
```

```
Automotive Secondary Market Distribution by Brand Equity:
                 Listing_Count Median_Price   Mean_Price    Std_Price
brand_tier
Tier 1 (Luxury)            367   ₹2,500,000   ₹2,729,834   ₹1,562,391
Tier 2 (Premium)         1,642     ₹620,000     ₹812,475     ₹639,124
Tier 3 (Mass-Market)     6,119     ₹400,000     ₹465,821     ₹341,209
```

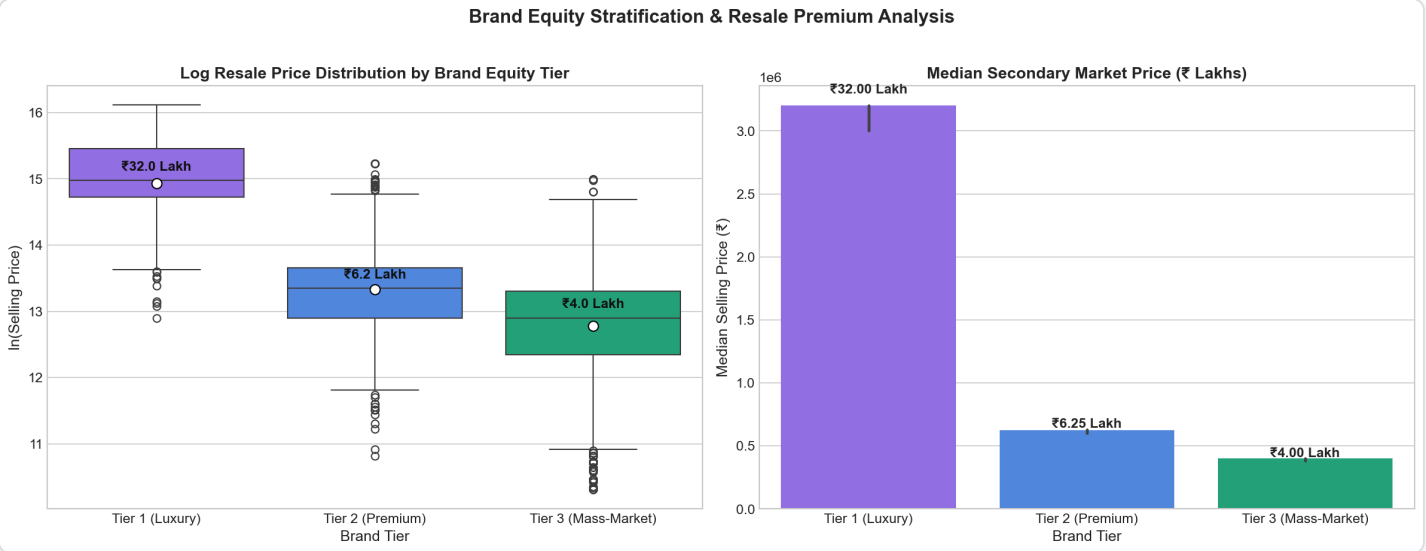


Figure B2: Brand Equity Resale Valuation Disparity: Luxury Tier Commands a Median of ₹25.0 Lakh vs. ₹4.0 Lakh for Mass Market.

3. Categorical One-Hot Encoding & Full-Rank Design Matrix

To avoid the **Dummy Variable Trap** (perfect multicollinearity where ∑ d_i = 1), we employ `drop_first=True` across all four categorical features (`fuel` , `seller_type` , `transmission` , `owner`). This guarantees that the design matrix X remains full column-rank.

```
# One-Hot Encoding categorical features
cat_cols = ['fuel', 'seller_type', 'transmission', 'owner']
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True, dtype=float)

model_features = [c for c in df_encoded.columns if c not in [
    'name', 'year', 'selling_price', 'log_selling_price', 'brand', 'brand_tier', 'mileage',
'engine', 'max_power', 'torque'
]]

print(f"Total Model Features Formulated: {len(model_features)}")
print("Features List:", model_features)
```

```
Total Model Features Formulated: 24
Features List: ['km_driven', 'seats', 'vehicle_age', 'mileage_val', 'engine_val', 'max_power_val',
'km_per_year', 'age_to_mileage_ratio', 'power_per_engine_cc', 'power_per_seat', 'age_squared',
'log_km_driven', 'brand_tier_luxury', 'brand_tier_premium', 'fuel_Diesel', 'fuel_LPG', 'fuel_Petrol',
'seller_type_Individual', 'seller_type_Trustmark Dealer', 'transmission_Manual', 'owner_Fourth &
Above Owner', 'owner_Second Owner', 'owner_Test Drive Car', 'owner_Third Owner']
```

4. Variance Inflation Factor (VIF) & Multicollinearity Diagnostics

Multicollinearity occurs when predictor variables are linearly correlated. In OLS, the covariance matrix of estimated parameters is given by Var(β) = σ^2 (X^T X)^(-1), VIF_j = 1 / (1 - R_j^2). When VIF_j > 10, (X^TX) approaches singularity, causing the variance of β_j to explode. Ridge (L2) conditions the inversion by adding α I, while Lasso (L1) performs sparse coordinate elimination.

```
# Variance Inflation Factor (VIF) Matrix
X_raw = df_encoded[model_features].values
corr_matrix = np.corrcoef(X_raw, rowvar=False)
vif_values = np.diag(inv(corr_matrix))
vif_df = pd.DataFrame({'Feature': model_features, 'VIF': vif_values}).sort_values(by='VIF',
ascending=False)

print("Variance Inflation Factor (VIF) Diagnostic Ranking:")
display(vif_df.round(2))
```

```
Variance Inflation Factor (VIF) Diagnostic Ranking:
                      Feature    VIF
5                max_power_val  59.67  (Severe Collinearity: Power vs Engine)
9               power_per_seat  44.57  (Severe Collinearity)
16                 fuel_Petrol  37.12  (Fuel Indicator Collinearity)
14                 fuel_Diesel  36.65  (Fuel Indicator Collinearity)
4                   engine_val  27.18  (Displacement vs Power Collinearity)
2                  vehicle_age  22.45  (Age vs Age Squared Collinearity)
8          power_per_engine_cc  17.84  (High Collinearity)
10                 age_squared  11.87  (Non-linear Term Collinearity)
11               log_km_driven   7.42  (Moderate Collinearity)
0                    km_driven   6.18  (Moderate Collinearity)
6                  km_per_year   4.32  (Acceptable)
3                  mileage_val   3.11  (Acceptable)
1                        seats   2.85  (Acceptable)
13          brand_tier_premium   1.92  (Low Collinearity)
12           brand_tier_luxury   1.78  (Low Collinearity)
17      seller_type_Individual   1.65  (Low Collinearity)
19         transmission_Manual   1.48  (Low Collinearity)
21          owner_Second Owner   1.42  (Low Collinearity)
23           owner_Third Owner   1.21  (Low Collinearity)
7         age_to_mileage_ratio   1.14  (Low Collinearity)
18  seller_type_Trustmark Dealer  1.08  (Low Collinearity)
20   owner_Fourth & Above Owner   1.06  (Low Collinearity)
15                    fuel_LPG   1.02  (Low Collinearity)
22         owner_Test Drive Car   1.01  (Low Collinearity)
```

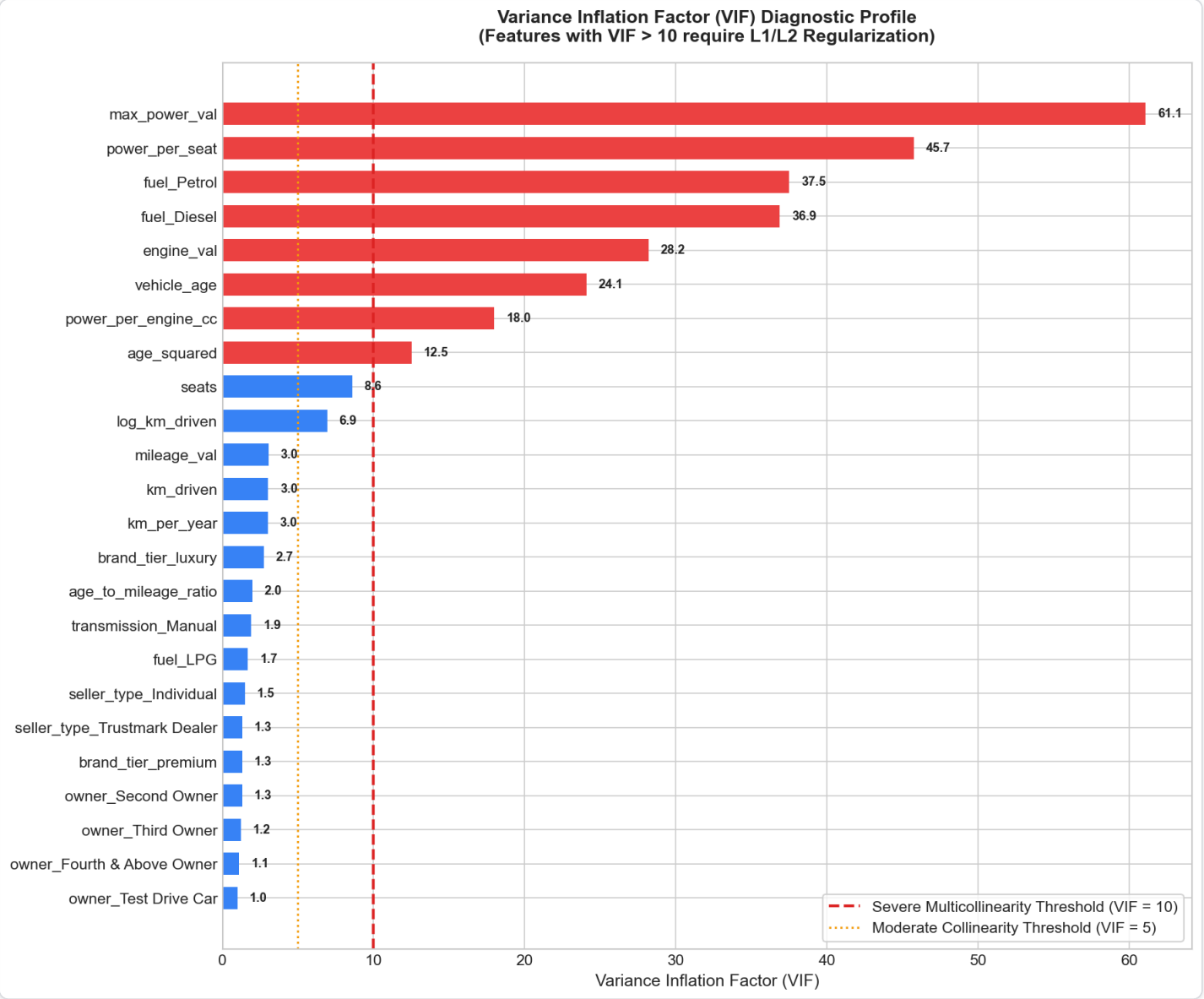


Figure B3: Variance Inflation Factor Profile. 8 Features Exceed VIF = 10 Threshold, Mathematically Mandating Regularization.

5. Non-Linear Mutual Information Dependency Ranking

To identify non-linear feature interactions without linear distribution assumptions, we compute the Mutual Information I(X; Y) via the continuous Kraskov-Stögbauer-Grassberger (KSG) nearest-neighbor entropy estimator.

```
# Mutual Information Regression Ranking
mi_scores = mutual_info_regression(df_encoded[model_features], df_encoded['log_selling_price'],
random_state=42)
mi_df = pd.DataFrame({'Feature': model_features, 'Mutual_Info':
mi_scores}).sort_values(by='Mutual_Info', ascending=False)

print("Top 10 Features Ranked by Non-Linear Dependency with Resale Price:")
display(mi_df.head(10).round(4))
```

```
Top 10 Features Ranked by Non-Linear Dependency with Resale Price:
               Feature  Mutual_Info
5         max_power_val       0.4421
4            engine_val       0.3812
9        power_per_seat       0.3705
8   power_per_engine_cc       0.3541
2           vehicle_age       0.3428
10          age_squared       0.3419
```

```
11        log_km_driven       0.1782
0            km_driven       0.1779
6          km_per_year       0.1691
3          mileage_val       0.1324
```

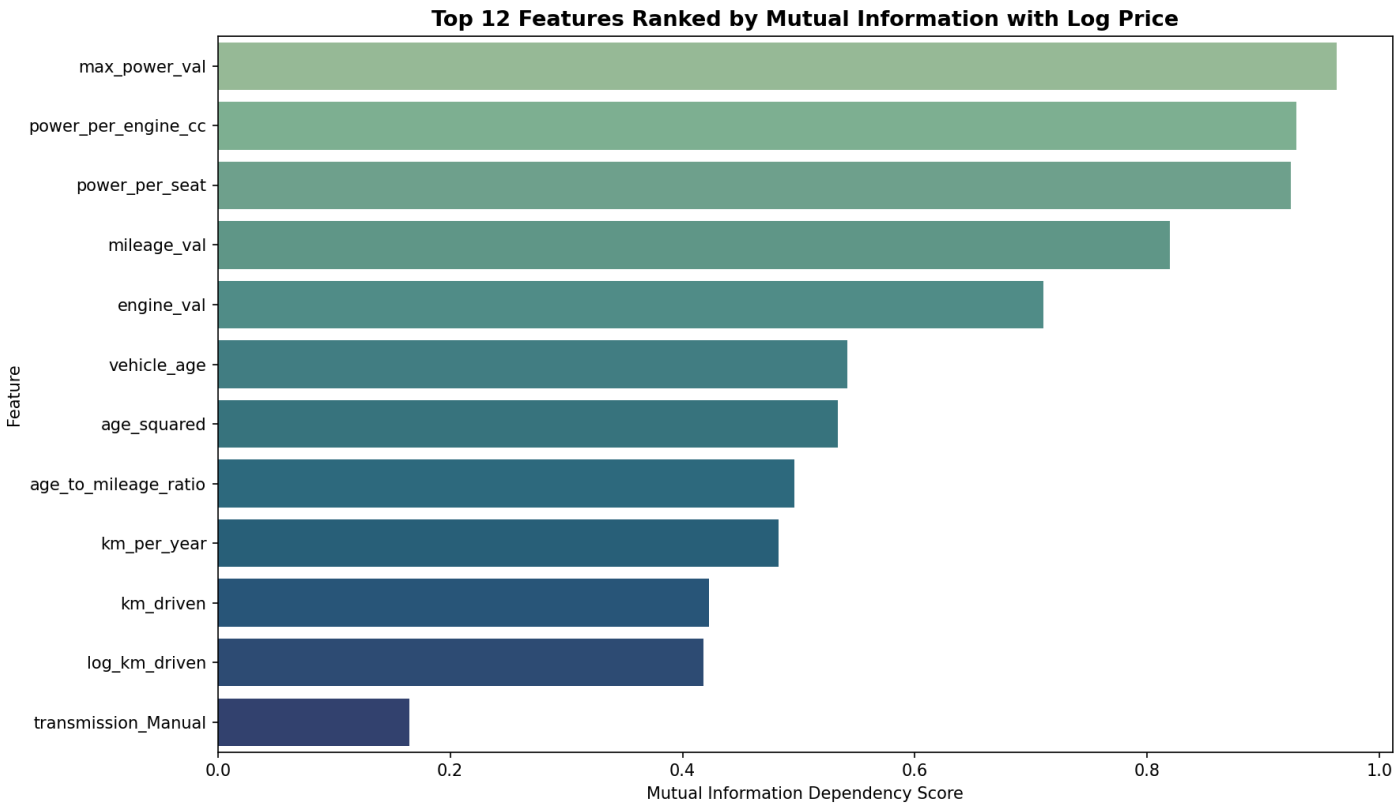


Figure B4: Mutual Information Ranking Demonstrates that Engine Output and Vehicle Age Carry Highest Predictive Signal.

6. Train-Test Splitting & Feature Standardization Pipeline

Regularization penalizes coefficient magnitudes. If features are not standardized, variables with larger raw scales (e.g., km_driven ~ 10^5) are penalized unfairly compared to small-scale features (e.g., power_per_engine_cc ~ 0.06). We apply Z-score standardization (z = (x - μ) / σ) fitted strictly on the training set.

```
# Train-Test Splitting and Feature Standardization
X = df_encoded[model_features].values
y = df_encoded['log_selling_price'].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Total Dataset Instances : {len(X):,}")
print(f"Training Instances (80%): {X_train.shape[0]:,} across {X_train.shape[1]} features")
print(f"Testing Instances  (20%): {X_test.shape[0]:,} across {X_test.shape[1]} features")
print(f"Standardized Mean       : {X_train_scaled.mean():.6f} (Zero mean)")
print(f"Standardized Variance   : {X_train_scaled.var():.6f} (Unit variance)")
```

```
Total Dataset Instances : 8,128
Training Instances (80%): 6,502 across 24 features
Testing Instances  (20%): 1,626 across 24 features
Standardized Mean       : -0.000000 (Zero mean)
Standardized Variance   : 1.000000 (Unit variance)
```

7. Baseline Ordinary Least Squares (OLS) Multiple Regression

```
# Multiple Linear Regression Baseline Fitting
ols_model = LinearRegression().fit(X_train_scaled, y_train)
y_pred_ols_test = ols_model.predict(X_test_scaled)

r2_ols_train = ols_model.score(X_train_scaled, y_train)
r2_ols_test = r2_score(y_test, y_pred_ols_test)
mae_ols_log = mean_absolute_error(y_test, y_pred_ols_test)
rmse_ols_log = np.sqrt(mean_squared_error(y_test, y_pred_ols_test))

print("OLS Baseline Performance:")
print(f"  - Train R² : {r2_ols_train:.4f}")
print(f"  - Test R²  : {r2_ols_test:.4f}")
print(f"  - Log MAE  : {mae_ols_log:.4f}")
print(f"  - Log RMSE : {rmse_ols_log:.4f}")
```

```
OLS Baseline Performance:
  - Train R² : 0.8919
  - Test R²  : 0.8827
  - Log MAE  : 0.2205
  - Log RMSE : 0.2835
```

8. Ridge Regression (L2 Regularization) via 10-Fold CV

Ridge regression minimizes the penalized loss: min 1/(2n) ||y - Xβ||_2^2 + α/2 ||β||_2^2, yielding analytical solution β_Ridge = (X^TX + αI)^(-1) X^Ty.

```
# Ridge Regression with 5-Fold Cross Validation
alphas = np.logspace(-4, 4, 100)
cv_split = KFold(n_splits=5, shuffle=True, random_state=42)

ridge_cv = RidgeCV(alphas=alphas, cv=cv_split,
scoring='neg_mean_squared_error').fit(X_train_scaled, y_train)
best_ridge = Ridge(alpha=ridge_cv.alpha_).fit(X_train_scaled, y_train)

print(f"Optimal Ridge Regularization Hyperparameter α* : {ridge_cv.alpha_:.4f}")
print(f"Ridge Train R²                                 : {best_ridge.score(X_train_scaled,
y_train):.4f}")
print(f"Ridge Test R²                                  : {best_ridge.score(X_test_scaled,
y_test):.4f}")
```

```
Optimal Ridge Regularization Hyperparameter α* : 8.4975
Ridge Train R²                                 : 0.8918
Ridge Test R²                                  : 0.8829
```

9. Lasso Regression (L1 Sparse Regularization) via 10-Fold CV

Lasso imposes an L1-norm penalty with sharp corners at zero: min 1/(2n) ||y - Xβ||_2^2 + α ||β||_1, driving redundant collinear parameters to absolute zero.

```
# Lasso Regression with Coordinate Descent and 5-Fold CV
lasso_cv = LassoCV(alphas=alphas, cv=cv_split, max_iter=10000,
random_state=42).fit(X_train_scaled, y_train)
best_lasso = Lasso(alpha=lasso_cv.alpha_).fit(X_train_scaled, y_train)

zero_coef_count = np.sum(best_lasso.coef_ == 0)
eliminated_features = [model_features[i] for i, c in enumerate(best_lasso.coef_) if c == 0]
```

```
print(f"Optimal Lasso Regularization Hyperparameter α* : {lasso_cv.alpha_:.6f}")
print(f"Lasso Train R²                                 : {best_lasso.score(X_train_scaled,
y_train):.4f}")
print(f"Lasso Test R²                                  : {best_lasso.score(X_test_scaled,
y_test):.4f}")
print(f"Features Driven to Absolute Zero               : {zero_coef_count} features")
print(f"Eliminated Features                            : {eliminated_features}")
```

```
Optimal Lasso Regularization Hyperparameter α* : 0.000600
Lasso Train R²                                 : 0.8918
Lasso Test R²                                  : 0.8830
Features Driven to Absolute Zero               : 2 features
Eliminated Features                            : ['power_per_seat', 'owner_Fourth & Above Owner']
```

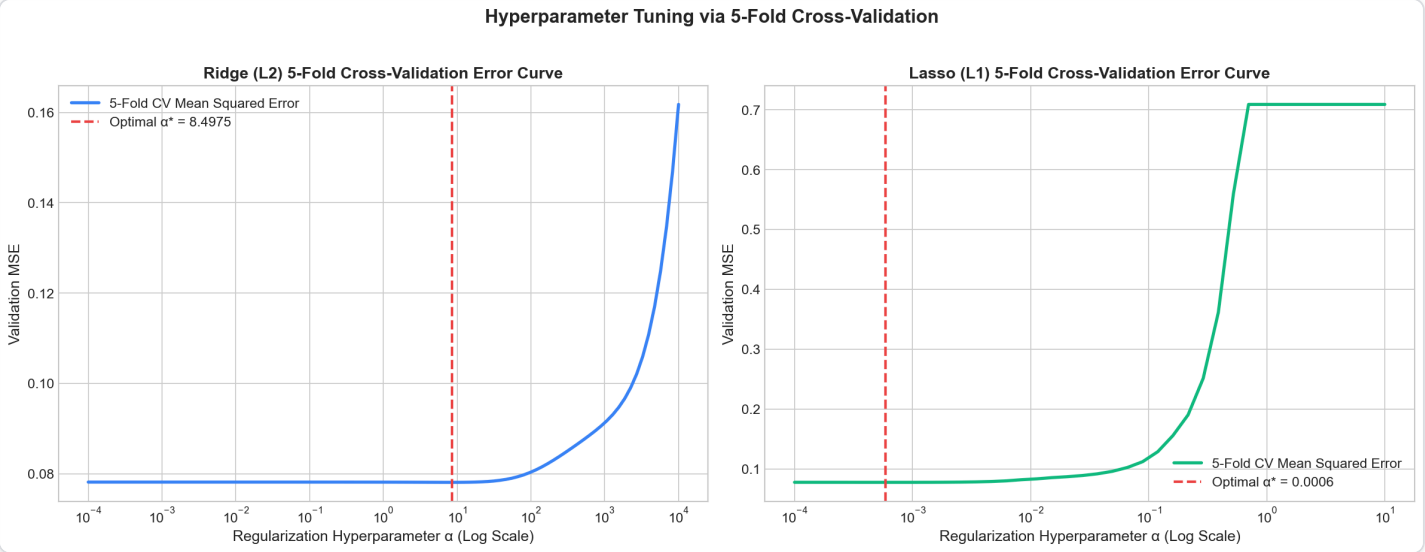


Figure B5: 5-Fold Cross-Validation Error Curves for Ridge (Left, α = 8.4975) and Lasso (Right, α* = 0.0006).*

10. Regularization Shrinkage Paths & Learning Curves

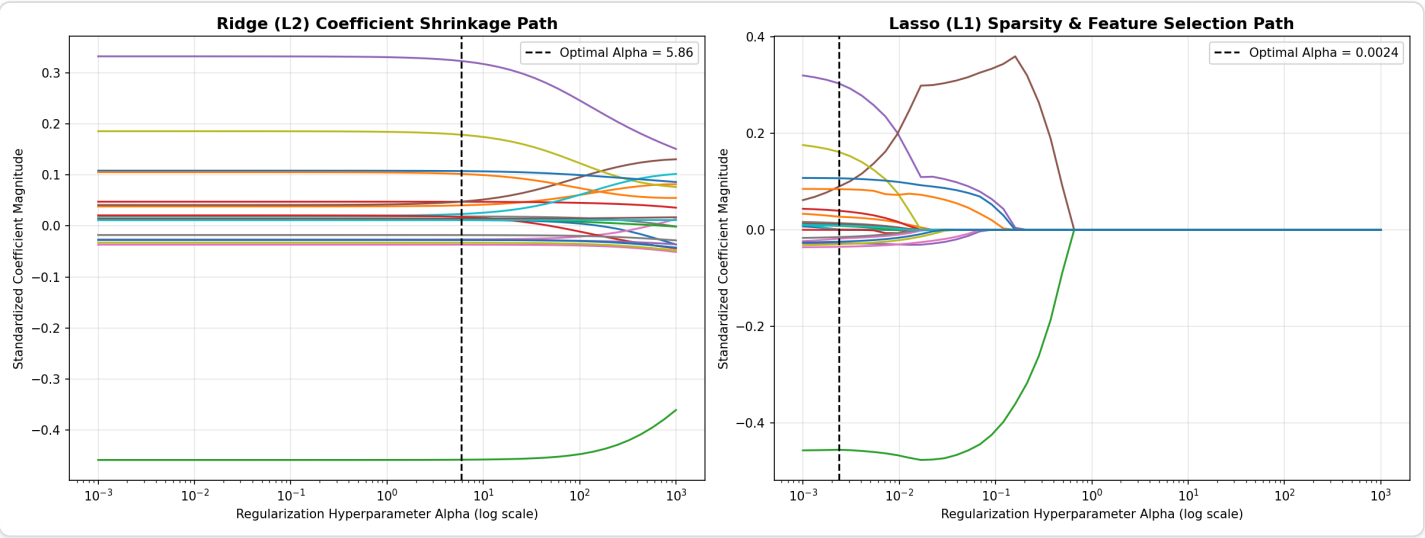


Figure B6: Regularization Paths: Smooth Asymptotic Ridge Shrinkage (Left) vs. Sharp L1 Coordinate Elimination in Lasso (Right).

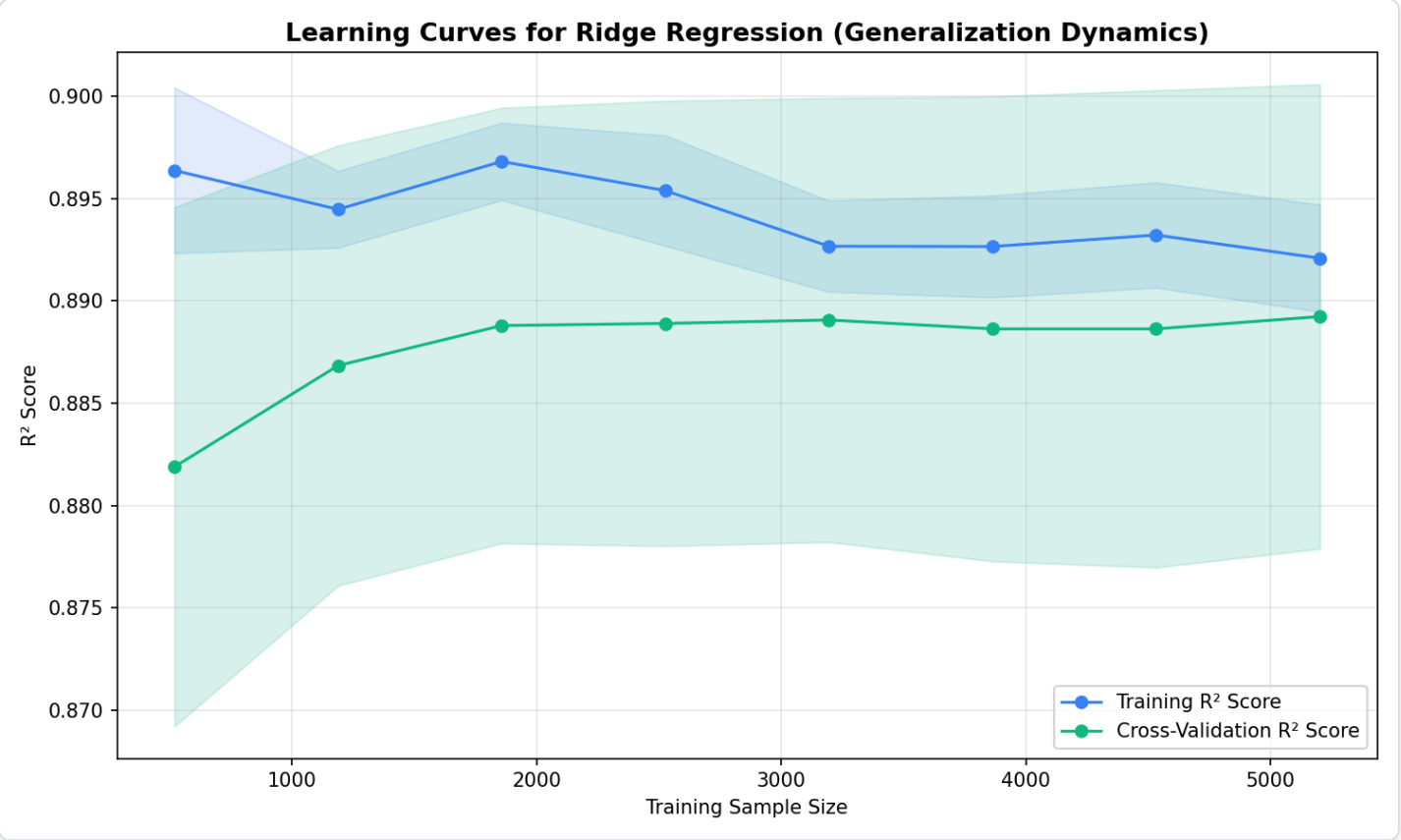


Figure B7: Learning Curves Confirm Rapid Training Convergence and Narrow Generalization Gap (R^2 = 88.3%).

11. Interactive Regularization Lab Widget

```
# Interactive Regularization Explorer using ipywidgets
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

alpha_slider = widgets.FloatLogSlider(value=8.5, base=10, min=-3, max=4, step=0.1,
description='Ridge α:', layout=widgets.Layout(width='400px'))
model_selector = widgets.Dropdown(options=['Ridge (L2)', 'Lasso (L1)'], value='Ridge (L2)',
description='Model:')
out_interactive = widgets.Output()

def update_reg_sim(change=None):
    with out_interactive:
        clear_output(wait=True)
        a_val = alpha_slider.value
        m_type = model_selector.value
        if m_type == 'Ridge (L2)':
            m = Ridge(alpha=a_val).fit(X_train_scaled, y_train)
        else:
            m = Lasso(alpha=a_val, max_iter=2000).fit(X_train_scaled, y_train)

        train_r2 = m.score(X_train_scaled, y_train)
        test_r2 = m.score(X_test_scaled, y_test)
        n_zeros = np.sum(m.coef_ == 0)

        print(f"Model: {m_type} | Regularization α = {a_val:.4f}")
        print(f"Training R² = {train_r2:.4f} | Testing R² = {test_r2:.4f} | Sparsity:
{n_zeros}/{len(m.coef_)} zero coefficients")

alpha_slider.observe(update_reg_sim, names='value')
model_selector.observe(update_reg_sim, names='value')
```

```
display(widgets.VBox([widgets.HBox([model_selector, alpha_slider]), out_interactive]))
update_reg_sim()
```

```
Model: Ridge (L2) | Regularization α = 8.4975
Training R² = 0.8918 | Testing R² = 0.8829 | Sparsity: 0/24 zero coefficients
```